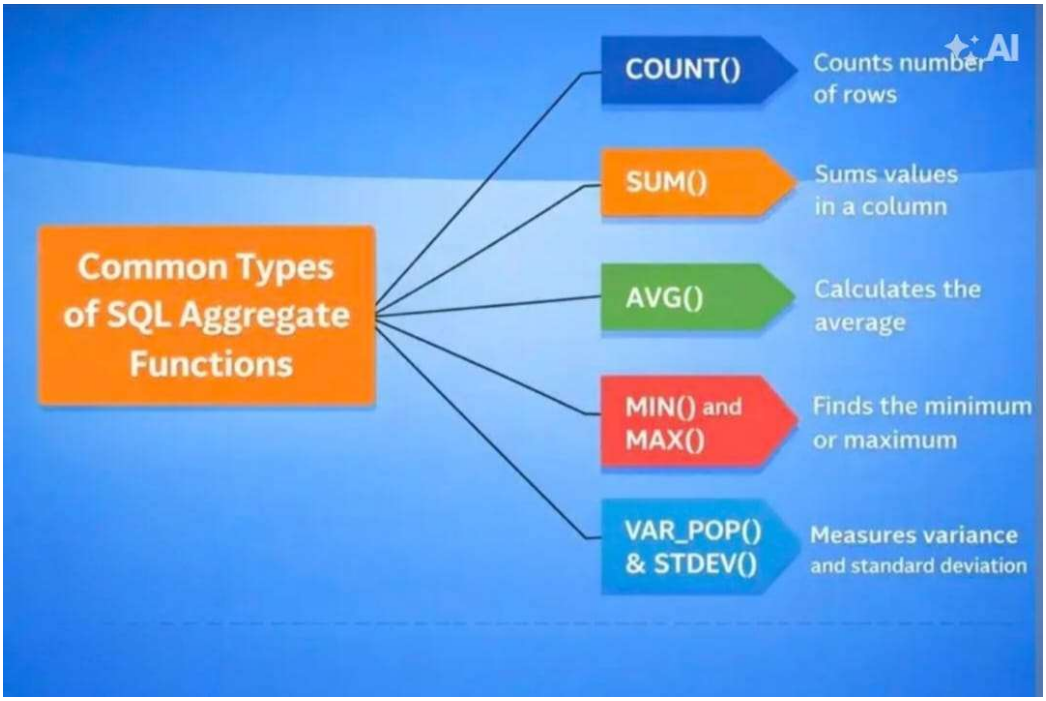
DBMS – SALES AND CUSTOMER ANALYTICS SYSTEM

Project Title: EcoCart – Sustainable Products E-Commerce Order Management System

1. Aggregate Functions

Definition

Aggregate functions are SQL functions used to perform calculations on a group of rows and return a single result.



1.1 COUNT()

Find the total number of orders:

```
SQL> SELECT COUNT(*) AS TOTAL_ORDERS
  2  FROM SAVANA_ORDERS;

TOTAL_ORDERS
------------
           4
```

1.2 SUM()

Find the total sales amount:

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES
  2  FROM SAVANA_ORDERS;

TOTAL_SALES
-----------
      46594
```

1.3 AVG()

Find the average order amount:

```
SQL> SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_AMOUNT
  2  FROM SAVANA_ORDERS;

AVERAGE_AMOUNT
--------------
       11648.5
```

1.4 MIN()

Find the lowest order amount:

```
SQL> SELECT MIN(TOTAL_AMOUNT) AS LOWEST_AMOUNT
  2  FROM SAVANA_ORDERS;

LOWEST_AMOUNT
-------------
         1598
```

1.5 MAX()

Find the highest order amount:

```
SQL> SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_AMOUNT
  2  FROM SAVANA_ORDERS;

HIGHEST_AMOUNT
--------------
         26998
```

2. Generate Total Sales Reports

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES
  2  FROM SAVANA_ORDERS;

TOTAL_SALES
-----------
      46594
```

2.1 Customer-wise Sales Report



```
SQL> SELECT CUSTOMER_ID,
  2         SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  3  FROM SAVANA_ORDERS
  4  GROUP BY CUSTOMER_ID
  5  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        101          42997
        102           1999
        103           1598
```

3. Find Top Customers Based on Purchase Amount

```
SQL> SELECT CUSTOMER_ID,
  2         SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  3  FROM SAVANA_ORDERS
  4  GROUP BY CUSTOMER_ID
  5  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        101          42997
        102           1999
        103           1598
```

4.Identify Best-Selling Products

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(D.QUANTITY) AS TOTAL_SOLD
  4  FROM SHOP_PRODUCT P
  5  JOIN SAVANA_ORDER_DETAILS D
  6  ON P.PRODUCT_ID = D.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY TOTAL_SOLD DESC;

PRODUCT_ID
----------
PRODUCT_NAME
----------------------------------------
---------
TOTAL_SOLD
----------
       201
T-Shirt
         1
```

4.1 Product Sales Amount

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(D.QUANTITY) AS TOTAL_QUANTITY,
  4         SUM(D.SUBTOTAL) AS TOTAL_SALES
  5  FROM SHOP_PRODUCT P
  6  JOIN SAVANA_ORDER_DETAILS D
  7  ON P.PRODUCT_ID = D.PRODUCT_ID
  8  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
---------
TOTAL_QUANTITY TOTAL_SALES
-------------- -----------
       201
T-Shirt
             1         999
```

5. Category-wise Sales Analysis

```
SQL> SELECT P.CATEGORY_ID,
  2         SUM(D.SUBTOTAL) AS TOTAL_SALES
  3  FROM SHOP_PRODUCT P
  4  JOIN SAVANA_ORDER_DETAILS D
  5  ON P.PRODUCT_ID = D.PRODUCT_ID
  6  GROUP BY P.CATEGORY_ID;

CATEGORY_ID TOTAL_SALES
----------- -----------
          2         999
```

6. Combined Business Sales Report

```
SQL> SELECT O.CUSTOMER_ID,
  2         O.ORDER_ID,
  3         P.PRODUCT_NAME,
  4         P.CATEGORY_ID,
  5         D.QUANTITY,
  6         D.SUBTOTAL
  7  FROM SAVANA_ORDERS O
  8  JOIN SAVANA_ORDER_DETAILS D
  9  ON O.ORDER_ID = D.ORDER_ID
 10  JOIN SHOP_PRODUCT P
 11  ON D.PRODUCT_ID = P.PRODUCT_ID;

CUSTOMER_ID   ORDER_ID
----------- ----------
PRODUCT_NAME
--------------------------------------------------
---------
CATEGORY_ID   QUANTITY   SUBTOTAL
----------- ---------- ----------
        101       1001
T-Shirt
          2          1        999
```